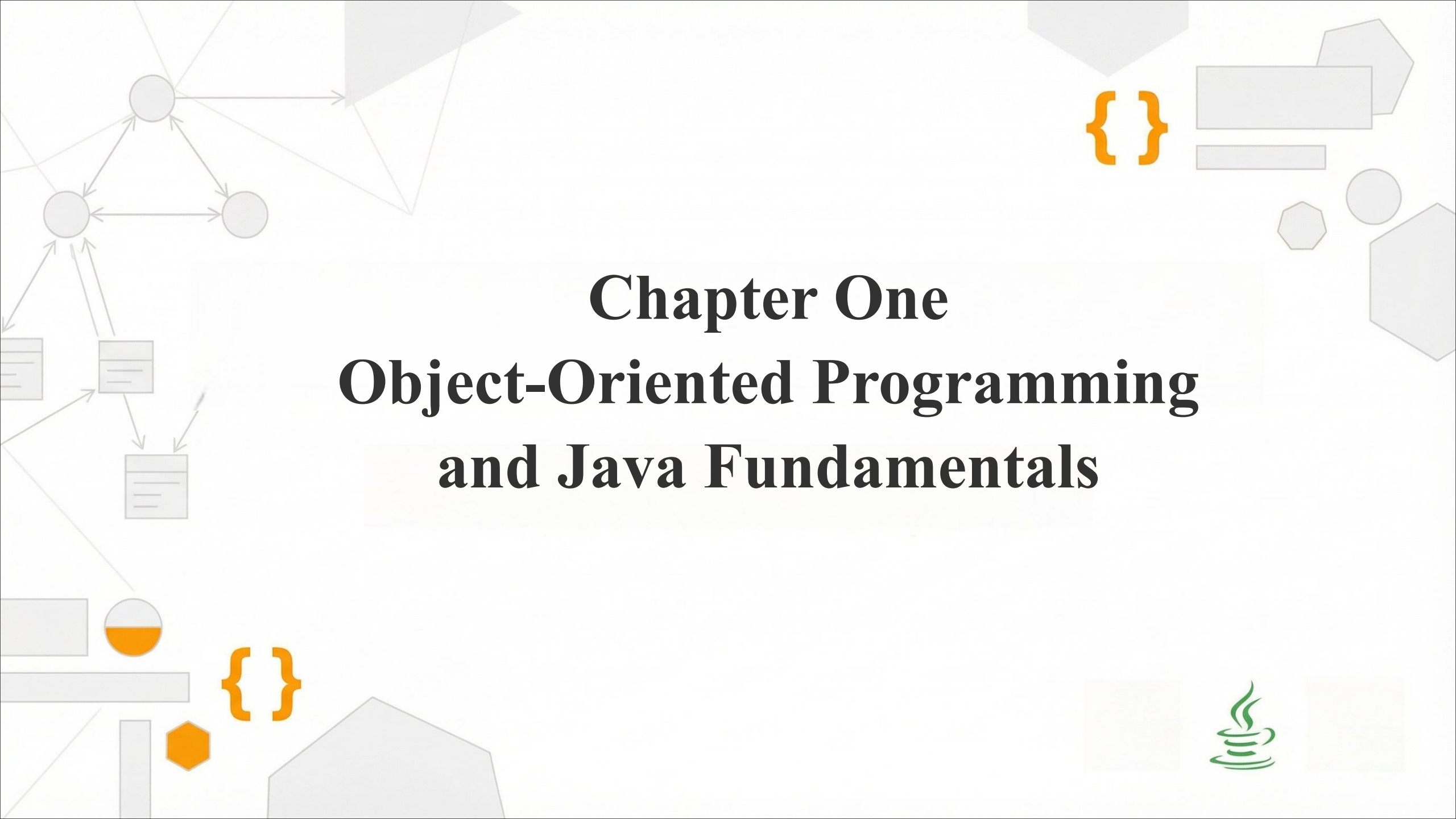


# Chapter One
# Object-Oriented Programming and Java Fundamentals

# Objectives: Your Learning Path

01

## Programming Paradigms

Understand and describe different programming approaches.

02

## OOP Principles

Explain the fundamental concepts of Object-Oriented Programming.

03

## Java Buzzwords

Discuss the key characteristics that define Java.

04

## Java Technology & Editions

Distinguish between various types of Java platforms.

05

## Java Development Tools

Understand basic tools and program types.

# Programming Paradigms

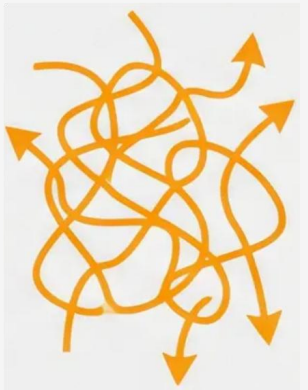
- Programming started with **binary code** and mechanical switches.
- High-level languages were developed with **English-like instructions** to simplify development.
- As computer capacity grew, developers began building increasingly **complex applications**.

**Expert programmers faced critical concerns:**

- How to avoid duplicate efforts and **reuse code**?
- How to control **global variables** in a shared environment?
- Difficult debugging (goto statements)
- How to maintain a **large code base** effectively

**Unstructured Programming**

- goto statements
  "Spaghetti code"
  Difficult to maintain
  Linear instruction flow



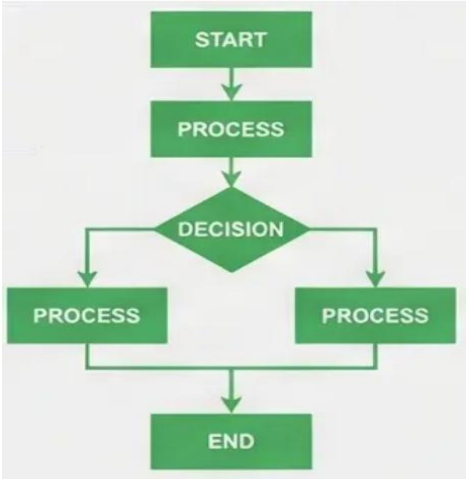
# Programming Paradigms …

## Structured Programming

- Introduced the concept of **functions** (procedures or subroutines).
- Each function was dedicated to solving one small, specific problem.
- Focus shifted to **managing interactions** between these functions.
- Global variables were largely replaced with **local variables** within functions.

### Structured Programming

- Control structures (if-else, loops)
  Functions and subroutines
  Improved readability
  Top-down approach




### Limitations of Structured Programming

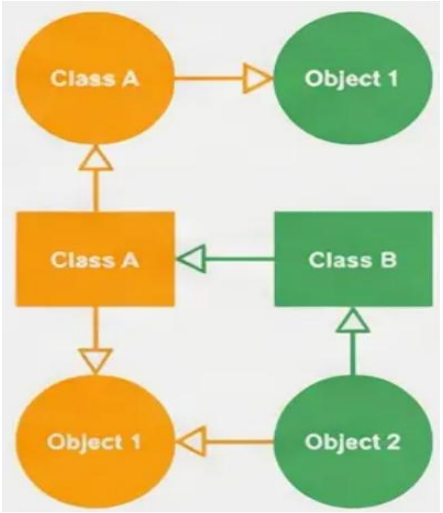
- Changing a **data type** requires updates across all functions in the application.
- Difficult to **model real-world scenarios** accurately.

# Programming Paradigms …

## Object-Oriented Programming

*Focus on data and objects.*

- Data + Methods = Single Unit
- Accurate real-world modeling
- Increased security and reuse




- Popular class-based OOP languages include **Java, Python, and C++**.
- Multiple independent objects can be created from the same class and interact together.

Evolution from complexity to modularity and maintainability

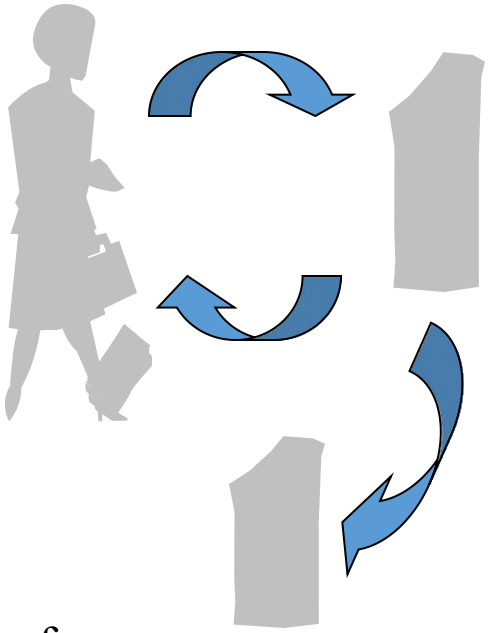
### Key Takeaway

- Evolution shifted the focus from **"How to do it"** (functions) to **"What it is"** (data/objects), leading to more robust and maintainable software systems.

# Structural vs. Object-Oriented

- **Structural**

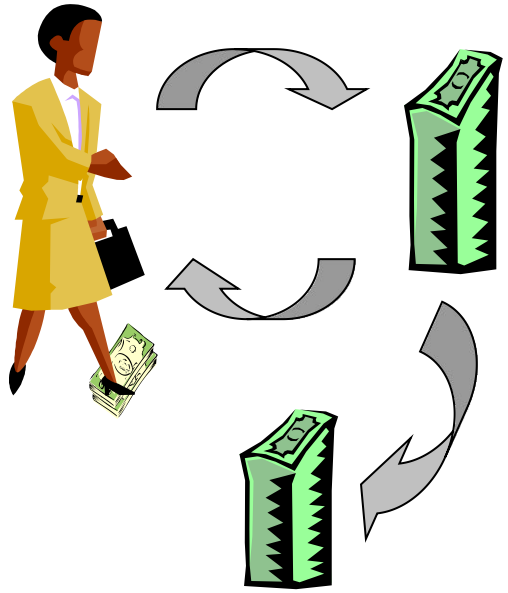
*Focus* on Actions



Withdraw, deposit, transfer

- **Object Oriented**

*Focus on Entities*



Customer, money, account

# Difference Summary

| Parameter | OOP | Procedural |
| --- | --- | --- |
| **Definition** | Models real-world environments using classes and objects | Collection of functions following a step-by-step approach |
| **Approach** | Divided into small chunks (objects) | Divided into small parts (functions) |
| **Security** | High (supports data hiding) | Lower security |
| **Importance** | Focus on Data | Focus on Functions |

# Object-Oriented Principles

## Class vs. Object: The Foundation of OOP

### The Class

"The Logical Template"

- Acts as a **blueprint** for objects
  - Define structure (attributes)
  - Defines behavior (methods)
- Consumes **no memory** until instantiated.

### The Object

"The Concrete Entity"

- An instance of a class
  - Has a unique **State** (values)
  - Has specific **Behavior** (actions)
- Possesses a distinct **Identity**.

**PARADIGM SHIFT**

- Moving from breaking problems into **functions** to decomposing them into **interacting objects**.

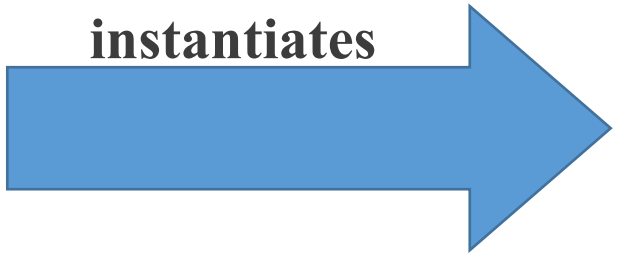
| CLASS | OBJECT |
| --- | --- |
| Student | Abel |

## Class

**Car Class**

- **Attributes**
  - **Color**
  - **Model**
  - **speed**
- **Methods:**
  - **start()**
  - **accelerate()**

**Defines structure and behavior**




## Object

**myCar Object**

- **Attributes**
  - **color = red**
    **model = Toyota**
    **speed = 0**
- **Available Methods:**
  - **start()**
  - **accelerate()**

**Contains real data and state**

**Key Insight:** This distinction enables code reuse and modular design – multiple objects can be created from a single class definition.

# Object-Oriented Principles…

## Encapsulation: Data Protection and Control

**The "Bundling" concept**

The mechanism of wrapping data (attributes) and code (methods) together into a single unit, forming a protective shield

### Data Protection

- Imposes restriction to prevent direct access to an object's internal state from outside the class.

### Data Hiding

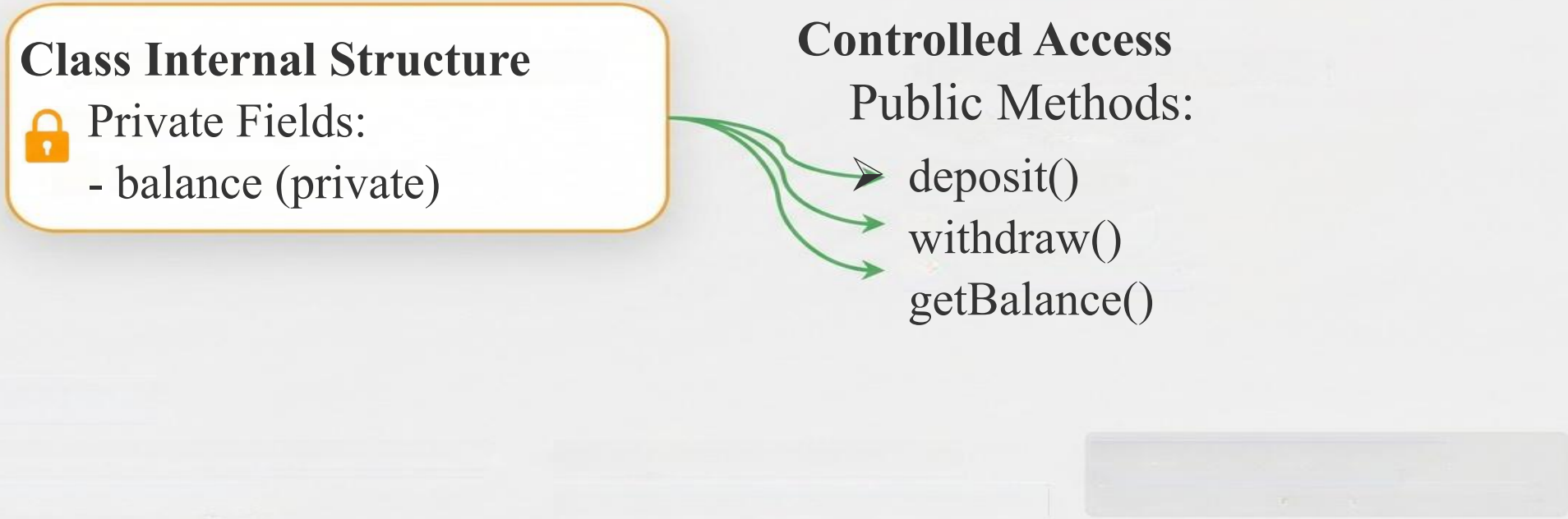
- Internal data is visible only through public methods (getters/setters), ensuring controlled modification.

### State Integrity

- Prevents external code from corrupting data by ensuring all changes go through validated logic

"Data is not allowed to flow freely throughout the system"

CLASS Student OBJECT Abel




## Key Benefits

- Prevents unauthorized data access
  Ensures data integrity
- Simplifies maintenance
  Provides controlled interface

**Example**:

```
BankAccount Class{
   private double balance;
   public void deposit(double
                  amount){ }
}
```

# Object-Oriented Principles…

## Abstraction: Simplifying Complexity

### Key Concept

- hides complex implementation details
- Exposes only essential features
- Achieved through interfaces and abstract classes
- Focus on “what” rather than “how
- Reduces complexity and increases efficiency

>_

## KEY BENEFITS

### Simplicity

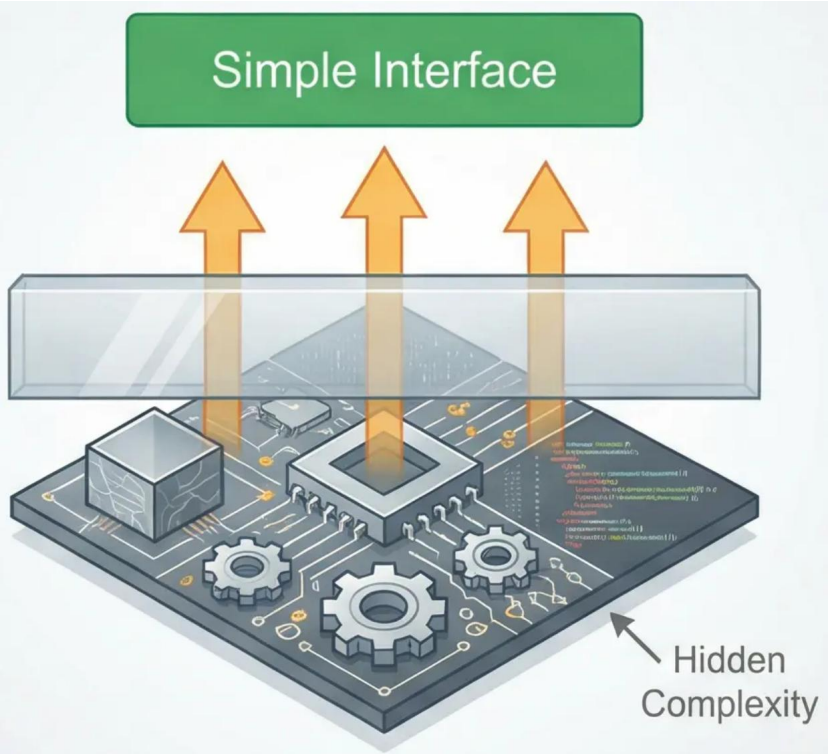
- Reduces complexity by presenting only relevant information to the system user.

### Code Isolation

- Internal changes do not affect external code, minimizing system-wide maintenance impact.

### Ease of Use

- Provides a high-level view that is easier to understand and interact with.




Complex Implementation

- Abstraction enables users to interact with complex systems through simplified interfaces

## Real-World Analogy: TV Remote

You only care about the **buttons (Interface)** to change channels. You don't need to understand the *internal circuits or signal frequencies (Implementation)*.

# Object-Oriented Principles…

## Inheritance: Hierarchical Code Reuse

### The IS-A Relationship

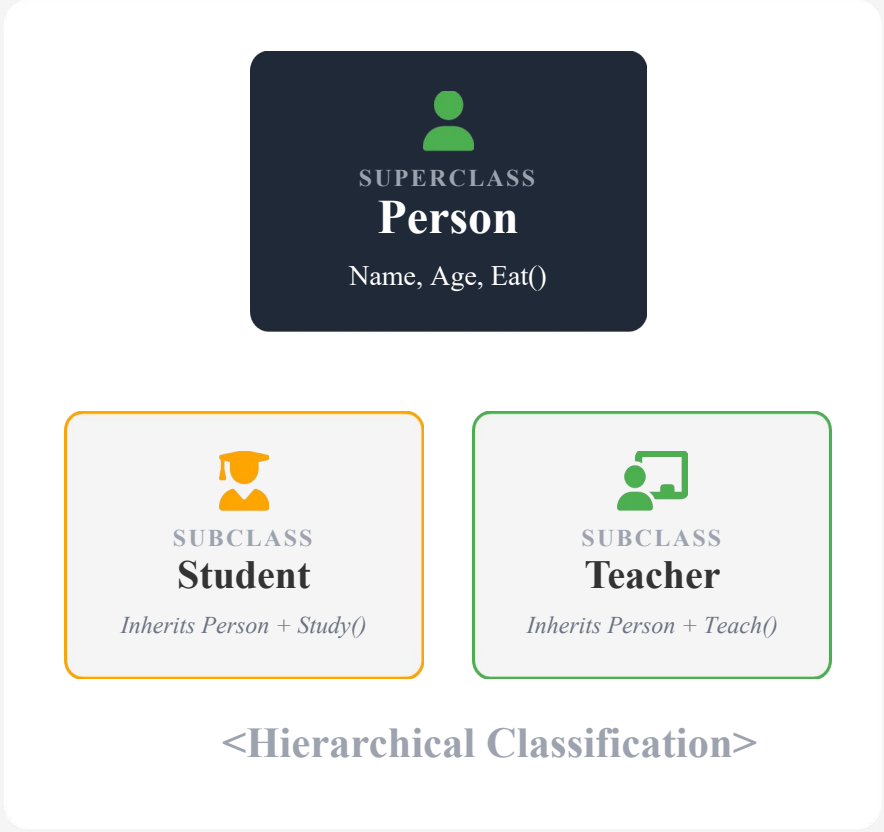
- Allows a subclass to acquire attributes and behaviors from a superclass, establishing a natural hierarchy.

### Extension & Overriding

- Subclasses can add unique methods or modify inherited ones to suit specific needs without altering the parent.

### Efficiency

- Significantly reduces duplicate code and centralizes maintenance within the parent class.




<Hierarchical Classification>

Changes in a parent class propagate to all child classes, simplifying large-scale maintenance.

## Key Benefits

- Code Reusability
- Hierarchical Relationships
- Reduced Duplication
- Easier Maintenance

# Polymorphism: One Interface, Many Forms

## Core Concept

- The ability of an object to take on **multiple forms**. A single interface (method name) can represent different underlying implementations.

EXAMPLE: OPERATOR OVERLOADING

Arithmetic Addition

**5 + 3** → **8**

String Concatenation

**"Hello" + "World"** → **"HelloWorld"**

> *"One interface, multiple methods.*
> *The system automatically selects the correct behavior based on the object's type."*

**Power of Polymorphism:** It simplifies code by allowing developers to write more general and reusable logic that adapts to specific object types at runtime.

Methods behave differently based on the object invoking them

## Method Overloading

**(Compile-time Polymorphism)**

- Same method name
  Different parameters
  Resolved at compile time

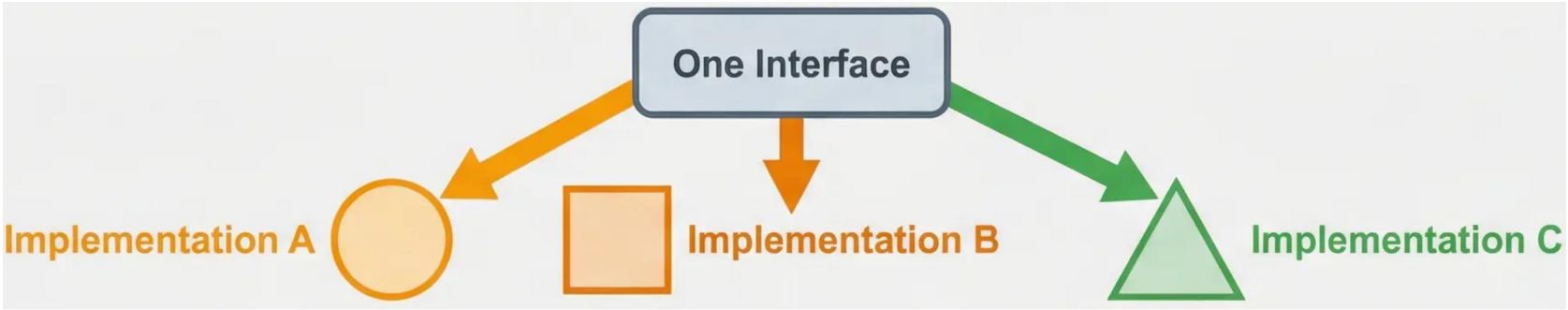
```
add(int a, int b)
add(double a, double b)
add(int a, int b, int c)
```

## Method Overriding

**(Runtime Polymorphism)**

- Parent-child relationship
  Same method signature
  Resolved at runtime

```
class Shape { draw() }
class Circle extends Shape {
    @Override draw() }
```




*Dynamic method dispatch enables flexible, extensible code*

# Java Buzzwords: Core Design Principles

## Simple

- Easy to learn; C-like syntax; Automatic Garbage Collection.

## Robust

- Strictly typed; Exception handling; No direct pointer manipulation., Automatic Memory Management

## Arch-Neutral

- "Write once, run anywhere" philosophy powered by the JVM.

## Secured

- Bytecode verifier; Security manager; Sandbox execution environment, built-in security

## Multithreaded

- Concurrent task support; Built-in multi-process synchronization.

## Distributed

- TCP/IP protocol support; Remote Method Invocation (RMI).

## Dynamic

- Runtime type information; Dynamic linking of code fragments.

## High Performance

- JIT compilation optimization

These principles collectively define Java's philosophy and widespread appeal

# Java Technology: Language and Platform

**Java Programming Language**

- Statically-typed, high-level syntax
  Automatic memory management
  Strong type checking & exception handling
- Inspired by C/C++ but simplified

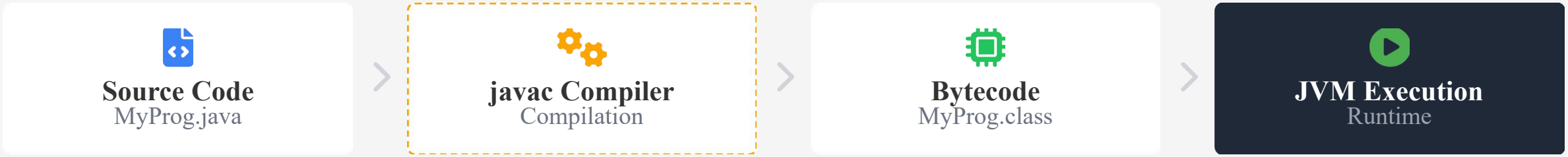
**Key Benefit**

**Write Once, Run Anywhere**

Cross-platform portability through bytecode and JVM

# JAVA TECHNOLOGY…

## Java Programming Language




## Platform Architecture

### JVM

- **The Virtual Machine:** A specification providing a runtime environment for bytecode. Platform-dependent implementation for "Write Once, Run Anywhere".

### JRE

- **Runtime Environment:** The physical implementation of JVM. Includes core libraries and supporting files needed to run applications.

### Java API

- **Library:** A massive collection of pre-written software components (packages) that insulate code from underlying hardware.

**WRITE ONCE, RUN ANYWHERE**

- *Java: Not just a language, but a complete ecosystem for cross-platform development.*

# Java Editions: Tailored for Different Environments

## Java SE (Standard Edition)

- Foundation of Java platform
  Core APIs and JVM
  Desktop and server applications
  General-purpose development

## Java EE (Enterprise Edition)

- Large-scale distributed systems
  Web services and transactions
  Banking and e-commerce platforms
  Enterprise applications

## Java ME (Micro Edition)

- Limited memory devices
  Mobile phones and IoT
  Embedded systems
  Lightweight applications

## JavaFX (Rich Client)

- Rich desktop applications
  Advanced graphics and media
  CSS styling support
  Modern user interfaces

Each edition targets specific deployment environments and application requirements

# Java Development Tools: From Compiler to IDE

## JDK (Java Development Kit)

- javac (Java compiler)
  java (Runtime launcher)
  jdb (Debugger)
  jar (Archive tool)
  javadoc (Documentation)

## JRE (Java Runtime Environment)

- Java Virtual Machine (JVM)
  Core libraries (java.lang, java.util)
  Supporting runtime tools
  Required to run Java apps

## Build Tools

- Maven (XML-based, enterprise)
  Gradle (Flexible, fast builds)
  Ant (Legacy, configurable)
  Dependency management

## Integrated Development Environments

- IntelliJ IDEA (Enterprise-focused)
  Eclipse (Open-source ecosystem)
  NetBeans (Lightweight, beginner friendly)
  VS Code (Cloud-native, extensions)

Integrated toolchain enables efficient Java development workflow

# Choosing the Right IDE

## IntelliJ IDEA

- Advanced code analysis
  Intelligent completion
  Enterprise integration
  Plugin ecosystem

Best for: Enterprise & large projects

## Eclipse

- Highly extensible
  Open-source community
  Multi-language support
  Robust debugging

Best for: Open-source & academic

## NetBeans

- Lightweight design
  GUI builder
  Maven integration
  Beginner-friendly

Best for: Education & small projects

## VS Code

- Fast & lightweight
  Extension support
  Cloud integration
  Minimal resources

Best for: Cloud & microservices

Choose your IDE based on project scale, team preferences, and deployment environment

# Java Program Types

### Stand-alone Apps

- CLI and GUI programs designed for specific tasks running directly on the OS.

### Java Applets

- Small programs embedded in HTML to provide interactive web browser features.

### Mobile Apps

- Applications developed for mobile devices using Java ME or the Android platform.

### Java Servlets

- Server-side components that generate dynamic web content and handle requests.

### Embedded/IoT

- Specialized systems like smart cards, industrial controllers, and IoT sensors.

# Types of Java Programs: Applications and Use Cases

## Standalone Applications

- GUI applications (JavaFX, Swing)
  Run locally on user's machine
  Media players, office tools

## Web Applications

- Server-side applications
  JSP, Servlets, Spring Boot
  Dynamic content via HTTP

## Mobile Applications

- Android development
  Legacy Java support
  Backend services

## Enterprise Applications

- Large-scale distributed systems
  Jakarta EE, EJB, JMS, JPA
  Banking, e-commerce, ERP